

PROYECTO SEMESTRAL — ENTREGA 1

**Asignación de vueltas a buses de una misma ruta con
ventanas de inicio y tiempos de viaje dependientes del
tráfico**

*Heurísticas List Scheduling y LPT para máquinas paralelas idénticas aplicadas al
transporte urbano de Cusco*

Asignatura	Algoritmos Avanzados
Docente	Héctor Eduardo Ugarte Rojas
Grupo	Grupo 4
Tema asignado	Optimización de asignación de trabajos en máquinas idénticas
Atributo del graduado	AG_C06 – Aprendizaje a lo largo de la vida
Hito	Entrega 1 – Cierre del primer periodo

Integrantes	Código
Aguilar Quispe, Yon	111276
Choque Mamani, Brayan Isau	215719
Huacani de la Cruz, Dany	081561
Moreano Villena, Miguel Angel	211859

Resumen

Este trabajo aborda la **asignación de vueltas a los buses de una misma ruta** del transporte urbano de Cusco, formulada como un problema de programación de trabajos en máquinas paralelas idénticas: cada bus es una máquina y cada vuelta origen–destino–origen es un trabajo. El objetivo es asignar cada vuelta a un bus de su propia flota y fijar su hora de salida para atender la mayor cantidad de vueltas y, entre esas soluciones, minimizar la carga máxima de conducción por bus ($L_{\max}$). El modelo respeta ventanas de inicio, una jornada de 06:00 a 22:00 y un almuerzo de dos horas por bus.

El tráfico se incorpora mediante un tiempo de viaje $p_i(s_i)$ que depende de la hora de salida s_i . Se calcula con un perfil de congestión por franjas horarias, con velocidad constante en cada franja, de modo que salir más tarde nunca implica llegar antes (propiedad FIFO). Se definen tres escenarios experimentales: sin tráfico (T0), perfil normalizado (T1) y perfil pesimista (T2). Los factores de congestión son parámetros definidos por el grupo y no mediciones oficiales.

Se implementaron en Python las heurísticas **List Scheduling (LS)** y **LPT** adaptadas a ventanas, almuerzo y tráfico, junto con un verificador independiente de restricciones y un conjunto de pruebas automatizadas. La experimentación preliminar usa 35 empresas/rutas de la fuente de referencia (4 733 vueltas) en los tres escenarios de tráfico.

Los resultados preliminares muestran que, sin tráfico, LS y LPT producen la misma solución. Con tráfico, LS obtiene mejores resultados en las instancias evaluadas: en el escenario T1 deja 155 vueltas sin atender frente a 293 de LPT, y su brecha media de $L_{\max}$ respecto de la cota inferior es 13.6 % frente a 19.4 %. Estos resultados indican que, cuando las vueltas tienen ventanas de inicio y duraciones dependientes de la hora, procesarlas en orden temporal es más adecuado que priorizar las de mayor duración. El prototipo es ejecutable y reproducible, y muestra la viabilidad del enfoque.

Palabras clave: máquinas paralelas idénticas, List Scheduling, LPT, tiempos de viaje dependientes del tiempo, ventanas de inicio, transporte urbano.

Índice

Resumen.....	1
1. Introducción.....	3
1.1 Contexto.....	3
1.2 Problema.....	3
1.3 Importancia de los tiempos de viaje dependientes de la hora.....	3
1.4 Objetivos.....	3
1.5 Metodología general.....	4
1.6 Alcance y delimitación.....	4
1.7 Organización del documento.....	4
2. Fundamentos y aprendizaje autónomo.....	6
2.1 Conceptos y algoritmos.....	6
2.2 Fuentes y criterio de selección.....	6
2.3 Conocimientos adicionales y aprendizaje autónomo.....	7
3. Stack tecnológico.....	8
4. Formulación del problema.....	9
4.1 Elementos del modelo.....	9
4.2 Origen de los datos.....	9
4.3 Modelo del tráfico.....	9

4.4 Entradas.....	10
4.5 Salidas.....	10
4.6 Restricciones.....	11
4.7 Supuestos.....	11
4.8 Formulación matemática y función objetivo.....	11
4.9 Datos e instancias.....	12
5. Diseño algorítmico.....	13
5.1 Estrategias de la Entrega 1.....	13
5.2 Estructuras de datos.....	13
5.3 Pseudocódigo.....	13
5.4 Invariantes y corrección.....	14
5.5 Complejidad.....	15
5.6 Mecanismos de referencia previstos.....	15
6. Diseño del sistema.....	16
7. Ejemplo manual.....	18
7.1 Datos.....	18
7.2 Cálculo del tiempo de viaje con tráfico.....	18
7.3 Asignación con List Scheduling.....	18
7.4 Asignación con LPT.....	19
7.5 Resultado.....	19
8. Plan de pruebas y validación.....	21
8.1 Validación de restricciones.....	21
8.2 Casos de prueba.....	21
8.3 Verificación de las salidas.....	21
9. Metodología experimental.....	23
9.1 Preguntas e hipótesis.....	23
9.2 Experimento E1: instancias, variables y métricas.....	23
9.3 Entorno y procedimiento reproducible.....	23
10. Prototipo y resultados preliminares.....	24
10.1 Prototipo.....	24
10.2 Resultados preliminares de E1.....	25
10.3 Contraste con las hipótesis.....	26
11. Discusión y limitaciones.....	27
11.1 Interpretación.....	27
11.2 Limitaciones y dificultades pendientes.....	27
12. Conclusiones.....	28
13. Referencias.....	29
Anexo A. Manual de instalación y uso.....	30
Anexo B. Contribuciones y bitácora.....	31
Anexo C. Resultados por ruta.....	32

1. Introducción

1.1 Contexto

El transporte urbano de Cusco opera mediante empresas que atienden una ruta fija con su propia flota. Cada día, la empresa debe decidir qué bus realiza cada vuelta y a qué hora sale, respetando la frecuencia de servicio, el descanso del personal y una jornada limitada. Una mala asignación concentra horas de conducción en algunos buses mientras otros quedan ociosos, y puede dejar sin cubrir salidas en las horas de mayor demanda (Ibarra-Rojas et al., 2015).

1.2 Problema

Dada una empresa/ruta con M buses idénticos y un conjunto de vueltas con ventana de inicio, se busca **asignar cada vuelta a un bus de su propia ruta y fijar su hora de salida**. La solución debe respetar la jornada de 06:00 a 22:00, el almuerzo de cada bus y la duración de cada vuelta según el tráfico. Se busca atender la mayor cantidad de vueltas y, entre esas soluciones, minimizar la carga de conducción del bus más cargado ($L_{\max}$).

Sin ventanas ni tráfico, el problema corresponde al problema clásico de programación en máquinas paralelas idénticas, que es NP-difícil (Garey y Johnson, 1979). Las ventanas de inicio, el almuerzo y el tráfico lo hacen más restrictivo y justifican el uso de heurísticas eficientes evaluadas experimentalmente.

1.3 Importancia de los tiempos de viaje dependientes de la hora

Una vuelta que sale en hora punta tarda más que la misma vuelta en horario valle. Si se supone una duración constante, se subestima la carga de los buses que trabajan en las puntas y se sobreestima la de los que trabajan en el valle; además, la hora de salida deja de influir en el resultado. Por eso el modelo representa la duración como una función $p_i(s_j)$ de la hora de salida, construida a partir de un perfil de congestión por franjas (Ichoua, Gendreau y Potvin, 2003).

1.4 Objetivos

Objetivo general

Desarrollar y evaluar experimentalmente una solución computacional basada en algoritmos de programación en máquinas idénticas para asignar las vueltas de las rutas de transporte urbano de Cusco, considerando ventanas de inicio, almuerzo y tiempos de viaje dependientes del tráfico.

Objetivos específicos

1. Formular el problema con entradas, salidas, restricciones, supuestos, función objetivo y un modelo explícito de tráfico.
2. Estudiar de forma autónoma los fundamentos de programación en máquinas paralelas y de tiempos de viaje dependientes del tiempo.
3. Implementar List Scheduling y LPT adaptados a ventanas, almuerzo y tráfico.
4. Validar la corrección mediante un verificador de restricciones, una instancia resuelta a mano y un plan de pruebas.
5. Comparar preliminarmente LS y LPT en las 35 rutas experimentales y en tres escenarios de tráfico.
6. Analizar críticamente los resultados, los supuestos y las limitaciones.

1.5 Metodología general

1. **Formulación:** modelo de máquinas idénticas con ventanas, almuerzo y tiempo de viaje $p_i(s_i)$.
2. **Datos:** construcción de instancias reproducibles a partir de los parámetros de 35 empresas/rutas y de supuestos experimentales explícitos.
3. **Diseño e implementación:** List Scheduling y LPT con inserción en huecos de la línea de tiempo de cada bus.
4. **Validación:** verificador independiente, cálculo manual y pruebas automatizadas.
5. **Experimentación preliminar:** comparación de LS y LPT sobre las 35 rutas en tres escenarios de tráfico.

La metodología global del proyecto contempla además, en etapas posteriores, la comparación con métodos exactos de referencia (Ramificación y Poda y CP-SAT) en instancias pequeñas y la evaluación de la robustez ante la variabilidad del tráfico. En la presente Entrega 1 se implementan y comparan List Scheduling y LPT.

1.6 Alcance y delimitación

Incluye:

- 35 empresas/rutas experimentales, cada una como instancia independiente;
- los buses de una misma ruta como máquinas idénticas;
- las vueltas completas origen → destino → origen como trabajos;
- ventanas de inicio para cada vuelta;
- una jornada de operación de 06:00 a 22:00;
- un periodo de almuerzo de 2 h por bus;
- tiempos de viaje dependientes de la hora mediante perfiles de tráfico experimentales;
- List Scheduling (LS);
- Longest Processing Time (LPT);
- un prototipo ejecutable;
- la validación de las soluciones;
- una experimentación inicial reproducible.

No incluye en esta etapa:

- el cambio de buses entre rutas;
- el ruteo de vehículos;
- datos de GPS en tiempo real;
- la simulación microscópica de tráfico (Lopez et al., 2018);
- el modelamiento de semáforos o de la infraestructura vial;
- Ramificación y Poda y CP-SAT completamente desarrollados;
- el análisis experimental completo del proyecto final.

1.7 Organización del documento

La sección 2 resume los fundamentos y el aprendizaje autónomo, y la 3 el stack tecnológico. La sección 4 formula el problema, la 5 presenta el diseño algorítmico de LS y LPT y la 6 el diseño del

sistema. La sección 7 resuelve una instancia pequeña paso a paso, la 8 presenta el plan de pruebas y la 9 la metodología experimental. La sección 10 muestra el prototipo y los resultados preliminares, la 11 los analiza y la 12 presenta las conclusiones de esta entrega.

2. Fundamentos y aprendizaje autónomo

2.1 Conceptos y algoritmos

Programación en máquinas paralelas idénticas. n trabajos con duración p_i se reparten entre m máquinas idénticas, y cada trabajo se ejecuta completo en una sola máquina (Pinedo, 2016). En la versión clásica se minimiza el instante en que termina la última máquina; como en esa versión no hay restricciones temporales, ese instante coincide con la carga de la máquina más cargada. Por eso, en este proyecto la medida de equilibrio es la **carga máxima** $L_{\max}$, definida en la sección 4.8.

List Scheduling (LS) recorre los trabajos en un orden dado y asigna cada uno a la máquina menos cargada. Sin restricciones temporales, su carga máxima es a lo sumo $(2 - 1/m)$ veces el óptimo (Graham, 1966). **LPT** (*Longest Processing Time first*) aplica la misma regla después de ordenar los trabajos de mayor a menor duración, y mejora la garantía a $(4/3 - 1/(3m))$ veces el óptimo (Graham, 1969). Estas garantías suponen que cualquier trabajo puede ejecutarse en cualquier momento; por eso no se trasladan directamente al problema con ventanas y tráfico y deben contrastarse experimentalmente.

Ventanas de inicio. Si cada trabajo solo puede iniciar en un intervalo $[r_i, d_i]$, el problema se aproxima a la programación de intervalos, donde ya es difícil decidir si todos los trabajos caben (Kolen et al., 2007). El almuerzo convierte cada bus en una máquina con un periodo de indisponibilidad (Lee, 1996).

Tiempos dependientes del tiempo y tráfico. Cuando la duración de un trabajo depende de su inicio (Gawiejnowicz, 2008), el tiempo de viaje puede calcularse dividiendo el día en franjas con velocidad constante e integrando la velocidad tramo por tramo (Ichoua, Gendreau y Potvin, 2003). El modelo cumple la **propiedad FIFO**: si $s_1 \leq s_2$, entonces $s_1 + p(s_1) \leq s_2 + p(s_2)$. Esta propiedad se usa en el diseño de los algoritmos (sección 5.4).

Mecanismos de referencia (etapa posterior). La Ramificación y Poda (Land y Doig, 1960) y la programación con restricciones de OR-Tools CP-SAT (Google, 2024) permiten obtener soluciones óptimas en instancias pequeñas. En el proyecto servirán como referencia para medir la calidad de LS y LPT; su uso se describe en la sección 5.6.

2.2 Fuentes y criterio de selección

Se usaron artículos con revisión por pares, libros de referencia y documentación oficial, seleccionados por su **pertinencia** para el modelo, su **confiabilidad** y la posibilidad de **verificar** sus resultados con los datos o las pruebas del proyecto.

Fuente	Uso en el proyecto
Graham (1966, 1969); Pinedo (2016); Garey y Johnson (1979)	Definición del problema, garantías de LS y LPT y complejidad.
Ichoua, Gendreau y Potvin (2003); Gawiejnowicz (2008)	Modelo de tráfico por franjas y propiedad FIFO.
Lee (1996); Kolen et al. (2007)	Modelado del almuerzo y de las ventanas de inicio.
Cormen et al. (2022)	Estructuras de datos de la línea de tiempo.
Land y Doig (1960); documentación de OR-Tools	Mecanismos de referencia previstos.
Tabla de flota operativa 2020	Parámetros de las rutas (documento de referencia, no oficial).

Tabla 1. Fuentes principales y su uso.

2.3 Conocimientos adicionales y aprendizaje autónomo

El proyecto requirió conocimientos que no formaban parte del contenido previo del grupo. La tabla resume qué se aprendió, para qué se usó y cómo se adquirió y verificó. Cada integrante registra sus actividades en una bitácora individual (Anexo B).

Conocimiento	Para qué se usó	Cómo se adquirió y verificó	Responsable
Tiempos de viaje por franjas y FIFO	Modelo de tráfico $p_i(s_i)$	Estudio de Ichoua et al.; cálculo manual; prueba automática de FIFO; hoja de cálculo de contraste	Aguilar
Garantías de LS y LPT	Diseño y análisis de las heurísticas	Graham y Pinedo; reproducción del peor caso de LPT en una prueba	Choque
Listas ordenadas con búsqueda binaria	Línea de tiempo de cada bus	Documentación de Python; Cormen et al.	Choque
Pruebas con pytest	Plan de pruebas	Documentación oficial; casos básicos, límite y adversos	Huacani
Validación de restricciones	Verificador independiente	Formulación del modelo; pruebas con soluciones alteradas	Huacani
Lectura y validación de datos con openpyxl/pandas	Instancias y Excel	Documentación oficial; regla general de vueltas verificada contra la hoja de trabajos	Moreano
Visualización con matplotlib	Diagramas de Gantt y gráficos	Documentación oficial	Moreano

Tabla 2. Conocimientos adicionales requeridos y forma de adquisición y verificación.

3. Stack tecnológico

Las herramientas se eligieron por su pertinencia para el problema, su instalación libre de licencias (reproducibilidad) y su integración con Python.

Tecnología	Función	Justificación
Python 3	Lenguaje de implementación	Lenguaje de referencia del curso; LS y LPT resuelven cada ruta en milisegundos.
Git / GitHub	Control de versiones	Historial del trabajo y de las contribuciones de cada integrante.
CSV / JSON	Instancias, perfiles de tráfico y resultados	Formatos abiertos, legibles y reproducibles.
openpyxl / pandas	Lectura del Excel del proyecto y análisis de resultados	Permiten leer los parámetros, validar los datos y generar tablas.
matplotlib	Gráficos y diagramas de Gantt	Figuras reproducibles de los resultados.
pytest	Pruebas automatizadas	Casos parametrizados y verificación continua.
Jupyter Notebook	Demostración del prototipo	Ejecución paso a paso (también en Google Colab).
OR-Tools (CP-SAT)	Prevista para la referencia exacta posterior	Libre; modela intervalos sin solapamiento y duraciones que dependen del inicio.

Tabla 3. Stack tecnológico.

4. Formulación del problema

4.1 Elementos del modelo

Elemento	Definición
Instancia	Una empresa/ruta. Las 35 rutas se resuelven por separado y los buses no cambian de ruta.
Máquina i	Un bus de la flota operativa de la ruta. Los buses son idénticos en velocidad; solo difieren en su turno de almuerzo.
Trabajo j	Una vuelta completa origen $\rightarrow$ destino $\rightarrow$ origen, con duración base $\bar{p}_j$ y ventana de inicio $[r_j, d_j]$.
Hora de salida s_j	Variable de decisión: instante en que inicia la vuelta j .
Tiempo de viaje $p_j(s_j)$	Duración real de la vuelta j si sale en s_j , según el perfil de tráfico (sección 4.3).
Almuerzo	Bloque de 120 min por bus que inicia en $[a_i, b_i]$: turnos 10:30–11:30, 12:00–13:00 y 13:30–14:30 (el bus k usa el turno $k \bmod 3$).
Carga L_i	Suma de los tiempos de viaje de las vueltas asignadas al bus i .

Tabla 4. Elementos del modelo.

4.2 Origen de los datos

Tipo	Contenido
Datos de la fuente de referencia	Flota operativa, tiempo de vuelta, demanda por hora y viajes por unidad vehicular (tabla de flota operativa 2020).
Parámetros construidos por el grupo	Excel del proyecto; vueltas planificadas = round(flota $\times$ viajes por unidad); inicios nominales equiespaciados; instancias JSON.
Supuestos experimentales	Ventanas de inicio (± 15 min en 06:00–09:00 y 16:00–19:00; ± 30 min en el resto); jornada de 06:00 a 22:00; almuerzo de 2 h en tres turnos; perfiles de tráfico T0, T1 y T2.

Tabla 5. Clasificación de los datos del modelo.

4.3 Modelo del tráfico

$p_j(s_j)$ es el tiempo de viaje (tiempo de procesamiento) de la vuelta j si sale a la hora s_j . El día se divide en franjas $[\tau_k, \tau_{k+1})$ con un factor de congestión $f_k > 0$: dentro de la franja k , un minuto real avanza $1/f_k$ minutos del recorrido base. Así, $p_j(s_j)$ es el menor Δ que cumple:

$$\int_{s_{\{j\}}}^{s_{\{j\}} + \Delta} dt / f(t) = \bar{p}_j$$

Como f es constante por tramos, la integral se resuelve franja por franja hasta completar $\bar{p}_j$ (pseudocódigo en la sección 5.3). Se definen tres perfiles:

Franja	Condición	T2 (pesimista)	T1 (normalizado)	T0 (sin tráfico)
06:00–06:30	Valle temprano	0.90	0.798	1
06:30–09:00	Punta mañana	1.30	1.152	1
09:00–12:00	Valle	1.00	0.886	1
12:00–14:30	Punta mediodía	1.20	1.064	1
14:30–17:00	Valle	1.00	0.886	1
17:00–20:00	Punta tarde	1.35	1.197	1
20:00–22:00	Valle nocturno	0.90	0.798	1
Media ponderada		1.128	1.000	1

Tabla 6. Factores de congestión por franja (parámetros experimentales). $T1 = T2 / 1.128$.



Figura 1. Perfiles de congestión T0, T1 y T2.

T1 se normaliza para que su media en la jornada sea 1. Así, la duración promedio del día coincide con el tiempo de vuelta de la fuente, que proviene de una velocidad media, y el tráfico solo redistribuye el tiempo entre puntas y valles. T2 representa un día de congestión generalizada (+12.8 % en promedio). T0 reproduce el caso sin tráfico.

Aclaración. Los factores de congestión son **parámetros experimentales** definidos por el grupo; no son mediciones oficiales del tráfico de Cusco. Se almacenan en archivos JSON y en el Excel del proyecto y pueden modificarse sin cambiar el código.

4.4 Entradas

- Ruta, empresa y flota M.
- Vueltas j con duración base $\bar{p}_j$, salida nominal y ventana $[r_j, d_j]$.
- Jornada [06:00, 22:00] y ventana de almuerzo de cada bus.
- Perfil de tráfico (T0, T1, T2 o un archivo JSON).
- Algoritmo a ejecutar (LS o LPT).

4.5 Salidas

- Bus asignado, hora de salida y hora de llegada de cada vuelta, o la marca NO_ASIGNADO.
- Hora de almuerzo de cada bus.
- Vueltas atendidas, $L_{\max}$, cota inferior y brecha, desbalance, utilización y sobre costo por tráfico.

- Resultado de la validación y tiempo de ejecución.

4.6 Restricciones

1. **Ruta:** una vuelta solo se asigna a un bus de su propia ruta.
2. **Asignación única:** cada vuelta se asigna a lo sumo a un bus; si no se asigna, queda como no atendida.
3. **Ventana:** $r_j \leq s_j \leq d_j$.
4. **Tiempo de viaje:** la vuelta termina en $C_j = s_j + p_j(s_j)$.
5. **Jornada:** $s_j \geq 06:00$ y $C_j \leq 22:00$.
6. **No solapamiento:** dos vueltas del mismo bus no se superponen.
7. **Almuerzo:** cada bus tiene un bloque de 120 min que inicia en su ventana y no se superpone con sus vueltas.

4.7 Supuestos

1. Los buses de una ruta son idénticos en velocidad.
2. La vuelta es indivisible y su duración base es el tiempo de vuelta de la fuente.
3. En una franja, el tráfico afecta por igual a toda la ruta.
4. Las ventanas y los turnos de almuerzo son políticas experimentales reproducibles.
5. No se consideran tiempos de terminal entre vueltas.

4.8 Formulación matemática y función objetivo

Conjuntos. $I = \{1, \dots, m\}$: buses de la ruta. $J = \{1, \dots, n\}$: vueltas de la ruta.

Parámetros. $\bar{p}_j$: duración base de la vuelta j ; $[r_j, d_j]$: ventana de inicio; $[a_i, b_i]$: ventana de inicio del almuerzo del bus i ; $f(t)$: factor de congestión del perfil; $p_j(s)$: tiempo de viaje de j si sale en s (sección 4.3); jornada $[06:00, 22:00]$.

Variables. $x_{ij} \in \{0,1\}$: vale 1 si la vuelta j se asigna al bus i ; $u_j \in \{0,1\}$: vale 1 si j no se atiende; s_j : hora de salida de j ; ℓ_i : inicio del almuerzo del bus i .

Restricción	Expresión
Asignación	$\sum_{i \in I} x_{ij} + u_j = 1 \quad \forall j \in J$
Ventana	$r_j \leq s_j \leq d_j \quad \forall j \in J$
Jornada	$s_j \geq 06:00; \quad s_j + p_j(s_j) \leq 22:00 \quad \forall j \in J$
Almuerzo	$a_i \leq \ell_i \leq b_i \quad \forall i \in I$
No solapamiento	los intervalos $[s_j, s_j + p_j(s_j))$ con $x_{ij} = 1$ y $[\ell_i, \ell_i + 120)$ son disjuntos $\forall i \in I$
Carga	$L_i = \sum_{j \in J} x_{ij} \cdot p_j(s_j); \quad L_{\max} = \max_{i \in I} L_i$

Tabla 7. Restricciones del modelo.

Función objetivo. En algunas rutas la flota no alcanza para todas las vueltas, de modo que el objetivo es **lexicográfico**:

1. **maximizar la cantidad de vueltas atendidas**, equivalente a minimizar $U = \sum_j u_j$;
2. **entre las soluciones con el mismo U , minimizar $L_{\max}$.**

$$\text{lex-min} (U , L_{\max})$$

Se usa $L_{\max}$ y no la hora de fin de la última vuelta porque las ventanas se reparten en toda la jornada: la última vuelta termina cerca de las 22:00 cualquiera sea la asignación, mientras que $L_{\max}$ sí mide cómo se reparte la conducción entre los buses.

Cota inferior. Sea $p_j^{\min}$ la menor duración posible de j dentro de su ventana. Para cualquier solución que atienda un conjunto S de vueltas, $L_{\max} \geq LB(S) = \max(\max_{j \in S} p_j^{\min}, \sum_{j \in S} p_j^{\min} / m)$. La brecha de una heurística se mide contra la cota de las vueltas que atiende.

4.9 Datos e instancias

Para la construcción de las instancias experimentales se seleccionaron 35 empresas/rutas de la fuente de referencia que presentan parámetros consistentes para la generación reproducible de las instancias del modelo. El grupo construyó un Excel con esos parámetros; la tabla de flota operativa 2020 es un documento de referencia y no un archivo entregado por las empresas.

Las vueltas de todas las rutas se generan con una misma regla general: el número de vueltas es $\text{round}(\text{flota} \times \text{viajes por unidad vehicular})$; los inicios nominales se reparten de forma equidistante entre las 06:00 y la última salida posible, $\text{round}(22:00 - \bar{p})$; y cada vuelta recibe una ventana de ± 15 min si su inicio nominal está entre 06:00–09:00 o 16:00–19:00, y de ± 30 min en el resto del día. Las vueltas resultantes se guardan en la hoja *Trabajos y ventanas* del Excel y en una instancia JSON por ruta.

Antes de resolver, el programa regenera las vueltas con esa regla y verifica que coincidan con la hoja del Excel. También comprueba que toda ventana cumpla $r_j \leq d_j$, que toda vuelta pueda terminar dentro de la jornada (con tolerancia de 1 min por el redondeo de las horas al minuto), que los identificadores sean únicos y que la flota no esté vacía. En total hay **4 733 vueltas** (entre 103 y 166 por ruta) y **1 080 buses** (entre 16 y 44 por ruta).

Para anticipar la dificultad de cada ruta se define la **saturación** de la flota:

$$\rho = \sum_j \bar{p}_j / (M \cdot 14 \text{ h})$$

Si $\rho > 1$, la ruta no puede atender todas sus vueltas ni siquiera sin tráfico. Es el caso de RTU-22: 19 buses y 127 vueltas de 2.13 h requieren 270.5 h, mientras la flota ofrece 266 h ($\rho = 1.02$). Las rutas siguientes en saturación son RTI-02 (0.87), RTU-05 (0.82) y RTU-01 (0.79).

5. Diseño algorítmico

5.1 Estrategias de la Entrega 1

Las dos estrategias de esta entrega, **List Scheduling (LS)** y **Longest Processing Time (LPT)**, comparten un esquema de lista con inserción en huecos. Se recorre una lista de vueltas en el orden propio de cada estrategia. Para cada vuelta se busca, en cada bus de la ruta, la salida factible más temprana que respete la ventana, la jornada y el almuerzo, y se elige el bus de menor carga acumulada. Si ningún bus puede atender la vuelta, esta queda registrada como no atendida.

Estrategia	Orden de la lista	Selección del bus	Desempate
List Scheduling (LS)	Inicio mínimo de ventana r_i (y salida nominal)	Bus factible de menor carga acumulada	Salida más temprana; luego índice de bus
LPT	Tiempo de viaje en la salida nominal, $p_i(\text{nominal})$, de mayor a menor	Igual que LS	Igual que LS

Tabla 8. Estrategias implementadas en la Entrega 1.

En LPT, la duración que define el orden se evalúa en la salida nominal de cada vuelta. Con tráfico, la vuelta "más larga" es la que cae en hora punta; sin tráfico, todas las vueltas de una ruta duran lo mismo y el orden de LPT se desempata por r_i , que es el orden de LS.

5.2 Estructuras de datos

Estructura	Implementación	Operaciones y costo
Job	Registro inmutable: id, ruta, $\bar{p}$, r , d , salida nominal	Acceso $O(1)$
Bus	Registro: id, ruta, ventana de almuerzo $[a, b]$, duración del almuerzo	Acceso $O(1)$
TrafficProfile	Lista ordenada de franjas (inicio, fin, factor) y factor fuera de franjas	<code>travel_time</code> en $O(F)$, con $F = 7$ franjas
BusTimeline	Listas ordenadas de inicios, fines y etiquetas de las vueltas del bus, más su carga	Recorrido de huecos $O(k)$; inserción con <code>bisect</code> $O(\log k)$ + desplazamiento $O(k)$
Schedule	Diccionario vuelta $\rightarrow$ (bus, salida, llegada), lista de no atendidas y almuerzos	Consulta $O(1)$
Instance	Vueltas, buses e índice por identificador	Lectura y escritura JSON $O(n + m)$

Tabla 9. Estructuras de datos (k = vueltas ya asignadas al bus).

Se usa una lista ordenada con búsqueda binaria en lugar de un árbol de intervalos porque cada bus realiza a lo sumo unas seis vueltas al día. Con k tan pequeño, el costo $O(k)$ es despreciable y el código es más fácil de verificar (Cormen et al., 2022).

5.3 Pseudocódigo

El pseudocódigo corresponde a las funciones `TrafficProfile.travel_time`, `BusTimeline.find_slot` y `greedy_schedule`, `list_scheduling` y `lpt` del módulo `algorithms/greedy.py`.

Tiempo de viaje $p_j(s)$

```

función TIEMPO_VIAJE(s,  $\bar{p}$ , franjas, f_fuera):
    restante  $\leftarrow \bar{p}$  ;  $t \leftarrow s$ 
    para cada franja ( $\tau_{ini}$ ,  $\tau_{fin}$ , f), en orden, con  $\tau_{fin} > s$ :
        si  $t < \tau_{ini}$ : ▷ tramo sin franja definida
            cap  $\leftarrow (\tau_{ini} - t) / f_{fuera}$ 
            si cap  $\geq$  restante: devolver  $t + restante \cdot f_{fuera} - s$ 
            restante  $\leftarrow$  restante - cap ;  $t \leftarrow \tau_{ini}$ 
        cap  $\leftarrow (\tau_{fin} - t) / f$  ▷ minutos base que caben
        si cap  $\geq$  restante: devolver  $t + restante \cdot f - s$ 
        restante  $\leftarrow$  restante - cap ;  $t \leftarrow \tau_{fin}$ 
    devolver  $t + restante \cdot f_{fuera} - s$  ▷ después de la última franja

```

Salida factible más temprana en un bus

```

función BUSCAR_HUECO(bus, j, p):
    para cada hueco libre [g0, g1] del bus, en orden cronológico:
        s0  $\leftarrow$  max( $r_j$ , g0) ; si s0 > d_j: devolver NINGUNO
        candidatos  $\leftarrow$  [s0]
        si max(g0, a_bus)  $\leq$  b_bus: ▷ el almuerzo podría ir en este hueco
            agregar max(s0, max(g0, a_bus) + 120) a candidatos
    para s en candidatos:
        si s > d_j o s + p(s) > g1: romper ▷ FIFO: más tarde tampoco cabe
    si el almuerzo aún cabe con [s, s + p(s)] ocupado: devolver (s, s + p(s))
    devolver NINGUNO

```

List Scheduling y LPT

```

función GREEDY(instancia, orden, perfil):
    para cada bus i:  $L_i \leftarrow 0$  ; línea_i  $\leftarrow$  vacía
    para cada vuelta j en orden:
        p  $\leftarrow t \mapsto$  TIEMPO_VIAJE(t,  $\bar{p}_j$ , perfil)
        mejor  $\leftarrow$  NINGUNO
        para cada bus i de la ruta:
            h  $\leftarrow$  BUSCAR_HUECO(i, j, p)
            si h  $\neq$  NINGUNO y ( $L_i$ , h.inicio, i) < clave(mejor): mejor  $\leftarrow$  (i, h)
        si mejor = NINGUNO: marcar j como NO_ASIGNADA
        si no: insertar j en línea_i ;  $L_i \leftarrow L_i + (h.fin - h.inicio)$ 
    para cada bus: ubicar el almuerzo en su inicio factible más temprano

LS(instancia, perfil): GREEDY(instancia, vueltas ordenadas por ( $r_j$ , nominal_j, id_j), perfil)
LPT(instancia, perfil): GREEDY(instancia, vueltas ordenadas por (-TIEMPO_VIAJE(nominal_j,  $\bar{p}_j$ ),  $r_j$ , id_j), perfil)

```

5.4 Invariantes y corrección

- **I1 — Factibilidad parcial.** Tras cada inserción, la línea de tiempo de cada bus cumple ventana, jornada y no solapamiento, y conserva un hueco donde cabe el almuerzo. BUSCAR_HUECO solo acepta candidatos que mantienen esta condición, así que el invariante se conserva por inducción sobre las inserciones. Al final, el almuerzo se ubica en ese hueco.

- **I2 — Salida más temprana.** Por la propiedad FIFO, $s \mapsto s + p(s)$ es no decreciente. Dentro de un hueco, el candidato más temprano termina antes que cualquier otro posterior; si ese no cabe, ninguno posterior cabe. Esto justifica el romper del pseudocódigo.
- **I3 — Conservación.** Toda vuelta termina asignada a exactamente un bus o registrada como no atendida.
- **I4 — Cota válida.** Cada vuelta dura al menos $p_j^{\min}$ y la carga total se reparte entre m buses, de modo que $LB(S) \leq L_{\max}$.

Idea de corrección. Por I1 e I3, la solución final satisface todas las restricciones de la sección 4.6. Esto se comprueba además en cada ejecución con un validador independiente (sección 8.1).

5.5 Complejidad

Componente	Tiempo	Espacio	Comentario
TIEMPO_VIAJE	$O(F)$	$O(1)$	$F = 7$ franjas
BUSCAR_HUECO	$O(k)$	$O(k)$	$k \approx n/m$ vueltas por bus
Ordenamiento (LS y LPT)	$O(n \log n)$	$O(n)$	LPT evalúa además $p_i(\text{nominal})$ en $O(n \cdot F)$
LS y LPT completos	$O(n \log n + n \cdot m \cdot k)$	$O(n + m)$	Con $k \approx n/m$ queda $O(n^2)$
Validador	$O(n \log n)$	$O(n)$	Ordena por bus y revisa vecinos

Tabla 10. Complejidad de los componentes de la Entrega 1.

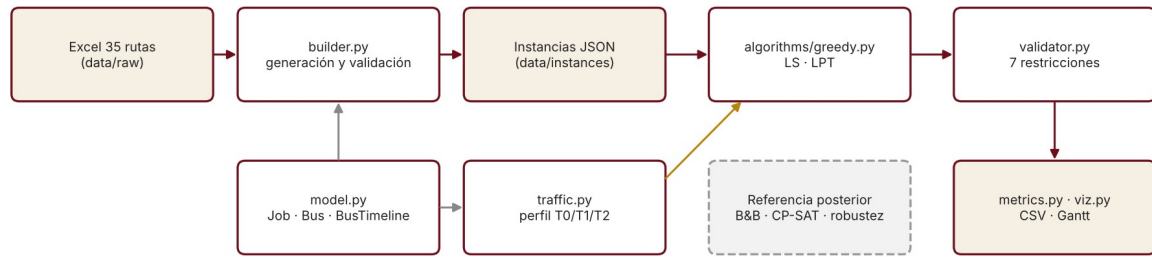
En las rutas experimentales (hasta 166 vueltas y 44 buses), LS y LPT resuelven cada ruta en unos 20 ms (sección 10.2).

5.6 Mecanismos de referencia previstos

El proyecto contempla adicionalmente **Ramificación y Poda** y **CP-SAT** como mecanismos de referencia para la validación de instancias pequeñas. En la presente Entrega 1 se implementan y comparan List Scheduling y LPT.

- **Propósito:** obtener la solución óptima de instancias pequeñas (del orden de 10 vueltas) para medir la distancia de LS y LPT al óptimo.
- **Ramificación y Poda:** enumera las asignaciones vuelta $\rightarrow$ bus y descarta las ramas cuya cota inferior, del tipo de $LB(S)$, no puede mejorar la mejor solución conocida (Land y Doig, 1960).
- **CP-SAT:** resuelve la formulación de la sección 4.8 con intervalos que no pueden solaparse y con duraciones que dependen de la hora de salida (Google, 2024).
- **Lugar en la metodología:** se usarán en la etapa de evaluación experimental para contrastar las heurísticas. El repositorio contiene prototipos de ambos, separados de LS y LPT, que no forman parte de la evaluación de esta entrega.

6. Diseño del sistema



Flechas guinda: datos de la instancia · doradas: modelo de tráfico · grises: estructuras compartidas · caja gris: etapa posterior

Figura 2. Módulos del sistema y flujo de información (la caja punteada corresponde a etapas posteriores).

El flujo es general para todas las rutas: datos de la ruta → parámetros del modelo → generación de vueltas → ventanas → tráfico → LS/LPT → validación → resultados. El Excel del proyecto se transforma en instancias JSON validadas; cada instancia se resuelve con el algoritmo y el perfil de tráfico elegidos, el validador revisa la solución y se generan métricas, archivos CSV y diagramas de Gantt.

Módulo	Responsabilidad
traffic.py	Perfiles T0/T1/T2 y cálculo de $p_i(s)$.
model.py	Vueltas, buses, instancia, línea de tiempo por bus (BusTimeline) y solución (Schedule).
builder.py	Lectura del Excel, regla general de generación de vueltas y ventanas, validación de datos, instancia del ejemplo manual.
algorithms/greedy.py	List Scheduling y LPT.
validator.py	Verificación independiente de todas las restricciones.
metrics.py	Vueltas atendidas, $L_{\max}$, cota inferior y brecha, desbalance, utilización, saturación p .
viz.py, cli.py	Diagramas de Gantt e interfaz de línea de comandos.
algorithms/bnb.py, algorithms/cpsat.py, simulation.py	Prototipos de los mecanismos de referencia y de robustez previstos para etapas posteriores.

Tabla 11. Módulos del paquete sched.

Formatos. Las instancias se guardan en JSON, una por ruta, con horas en formato HH:MM y duraciones en minutos:

```

{"route": "RTI-01", "company": "E.T. SAYLLA S.A.", "day_start": "06:00", "day_end": "22:00",
 "buses": [{"id": "RTI-01-B1", "lunch_r": "10:30", "lunch_d": "11:30", "lunch_min": 120}, ...],
 "jobs": [{"id": "RTI-01-J001", "base_min": 148.8, "r": "06:00", "d": "06:15", "nominal": "06:00"}, ...]}
  
```

Los perfiles de tráfico usan JSON con franjas (inicio, fin, factor). La salida de cada ejecución es un CSV con una fila por vuelta: ruta, vuelta, bus, ventana, salida, llegada, duración con tráfico y

duración base; las vueltas no atendidas llevan bus = N0_ASIGNADO. Todas las rutas de archivo son relativas al repositorio.

7. Ejemplo manual

7.1 Datos

Se usa una ruta pequeña con **3 buses** y **8 vueltas** de duración base **90 min** (cercana a la de RTI-08, 1.57 h, redondeada para calcular a mano). Los almuerzos inician en: B1 entre 10:30 y 11:30, B2 entre 12:00 y 13:00, y B3 entre 13:30 y 14:30. Se usa el perfil de tráfico **T2**, que tiene factores redondos.

Vuelta	Duración base $\bar{p}_j$	Ventana de inicio	p_j si sale en r_j	p_j si sale en d_j	p_j en la salida nominal
J1	90	06:00–06:30	103.7	117.0	110.3
J2	90	06:30–07:00	117.0	117.0	117.0
J3	90	07:00–07:30	117.0	110.8	114.2
J4	90	08:00–08:30	103.8	96.9	100.4
J5	90	09:30–10:30	90.0	90.0	90.0
J6	90	11:30–12:30	102.0	108.0	108.0
J7	90	16:30–17:00	111.0	121.5	116.3
J8	90	17:00–17:30	121.5	121.5	121.5

Tabla 12. Vueltas de la instancia manual y su tiempo de viaje (min) con tráfico T2.

7.2 Cálculo del tiempo de viaje con tráfico

J1 saliendo a las 06:00. La franja 06:00–06:30 ($f = 0.9$) dura 30 min reales y cubre $30/0.9 = 33.33$ min base. Faltan 56.67 min base, que en la punta mañana ($f = 1.3$) toman $56.67 \times 1.3 = 73.67$ min. Entonces $p_{J1}(06:00) = 30 + 73.67 = \mathbf{103.67 \text{ min}}$ y la llegada es a las 07:43.7.

J4 saliendo a las 08:00. Hasta las 09:00 hay 60 min reales a $f = 1.3$, que cubren 46.15 min base; los 43.85 restantes van a $f = 1.0$. Entonces $p_{J4}(08:00) = 60 + 43.85 = \mathbf{103.85 \text{ min}}$.

J4 saliendo a las 08:27. Hasta las 09:00 hay 33 min reales, que cubren 25.38 min base; los 64.62 restantes van a $f = 1.0$. Entonces $p_{J4}(08:27) = 33 + 64.62 = \mathbf{97.62 \text{ min}}$. Salir más tarde dentro de la ventana reduce el tiempo de viaje porque se pasa menos tiempo en la punta.

J7 saliendo a las 16:30. Hasta las 17:00 hay 30 min a $f = 1.0$; los 60 min base restantes van a $f = 1.35$ y toman 81 min. Entonces $p_{J7}(16:30) = \mathbf{111.0 \text{ min}}$.

Verificación. La hoja *Calculadora vuelta* del Excel reproduce este cálculo con fórmulas (06:00, 90 min, T2 $\rightarrow$ 103.67 min) y coincide con el programa.

7.3 Asignación con List Scheduling

Orden de la lista: J1, J2, ..., J8 (por inicio de ventana). En cada paso se muestran, para cada bus, la salida y llegada factibles y la carga previa (min).

Paso	Vuelta	B1	B2	B3	Elegido
1	J1	06:00–07:44 · 0	06:00–07:44 · 0	06:00–07:44 · 0	B1 (empate → índice)
2	J2	infactible	06:30–08:27 · 0	06:30–08:27 · 0	B2
3	J3	infactible	infactible	07:00–08:57 · 0	B3
4	J4	08:00–09:44 · 103.7	08:27–10:05 · 117.0	infactible	B1 (menor carga)
5	J5	09:44–11:14 · 207.5	09:30–11:00 · 117.0	09:30–11:00 · 117.0	B2 (empate → índice)
6	J6	12:30–14:18 · 207.5 (*)	infactible	11:30–13:12 · 117.0	B3
7	J7	16:30–18:21 · 207.5	16:30–18:21 · 207.0	16:30–18:21 · 219.0	B2 (207.0 < 207.5)
8	J8	17:00–19:02 · 207.5	infactible	17:00–19:02 · 219.0	B1

Tabla 13. Traza de List Scheduling (T2). (*) B1 debe iniciar su almuerzo entre 10:30 y 11:30, por eso J6 solo cabe después del almuerzo.

7.4 Asignación con LPT

Ordenando por tiempo de viaje en la salida nominal (última columna de la tabla de datos), el orden es J8 (121.5), J2 (117.0), J7 (116.3), J3 (114.2), J1 (110.3), J6 (108.0), J4 (100.4) y J5 (90.0).

Paso	Vuelta	Elegido	Salida–llegada	$p_i(s_i)$	Motivo
1	J8	B1	17:00–19:02	121.5	Todos vacíos → índice
2	J2	B2	06:30–08:27	117.0	B2 y B3 con carga 0
3	J7	B3	16:30–18:21	111.0	B1 ocupado por J8; B3 con carga 0
4	J3	B3	07:00–08:57	117.0	B3 (111.0) < B1 (121.5); B2 ocupado
5	J1	B1	06:00–07:44	103.7	Único factible
6	J6	B1	12:30–14:18	108.0	B1 (225.2) < B3 (228.0)
7	J4	B2	08:27–10:05	97.6	B2 (117.0) es el de menor carga
8	J5	B2	10:05–11:35	90.0	B1 infactible; B2 (214.6) < B3 (228.0)

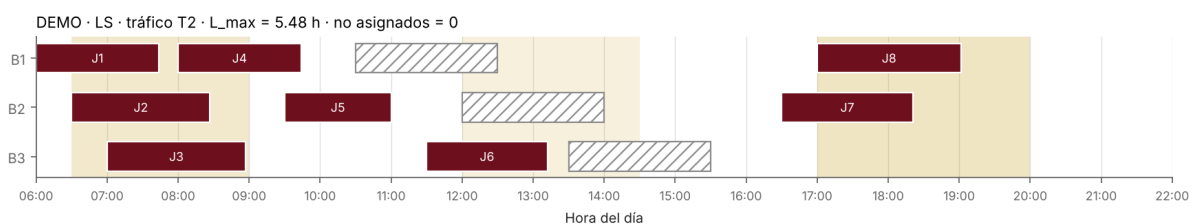
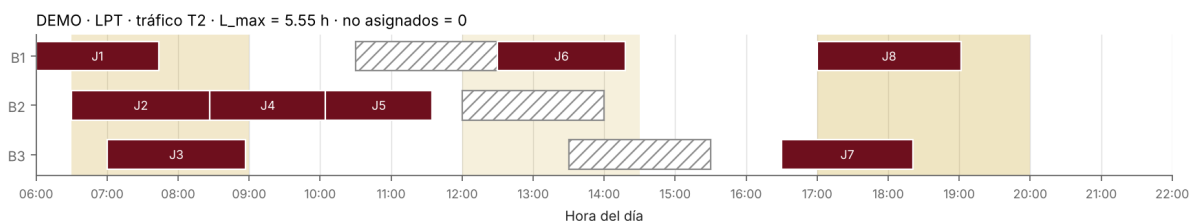
Tabla 14. Traza de LPT (T2).

7.5 Resultado

Vuelta	Ventana	LS: bus	LS: salida	LS: p_i	LPT: bus	LPT: salida	LPT: p_i
J1	06:00–06:30	B1	06:00	103.7	B1	06:00	103.7
J2	06:30–07:00	B2	06:30	117.0	B2	06:30	117.0
J3	07:00–07:30	B3	07:00	117.0	B3	07:00	117.0
J4	08:00–08:30	B1	08:00	103.8	B2	08:27	97.6
J5	09:30–10:30	B2	09:30	90.0	B2	10:05	90.0
J6	11:30–12:30	B3	11:30	102.0	B1	12:30	108.0
J7	16:30–17:00	B2	16:30	111.0	B3	16:30	111.0
J8	17:00–17:30	B1	17:00	121.5	B1	17:00	121.5

Tabla 15. Asignación final de cada vuelta (p_j en minutos).

Algoritmo	Carga B1	Carga B2	Carga B3	Vueltas atendidas	$L_{\max}$ (min)	Brecha vs LB
LS	329.0	318.0	219.0	8 / 8	329.0	+15.7 %
LPT	333.2	304.6	228.0	8 / 8	333.2	+17.2 %
Cota inferior LB					284.3	—
Sin tráfico (T0): LS = LPT				8 / 8	270.0	+12.5 % (LB = 240)

Tabla 16. Resultado de la instancia manual (cargas en minutos).**Figura 3.** Programa de List Scheduling (T2). Fondo dorado: franjas de congestión; rayado: almuerzo.**Figura 4.** Programa de LPT (T2).

Ambas estrategias atienden las 8 vueltas. Sin tráfico obtienen la misma carga máxima (270 min, con 3 + 3 + 2 vueltas de 90 min). Con tráfico, LS obtiene una carga máxima menor (329.0 frente a 333.2 min). LPT toma primero las vueltas largas de la tarde y reparte las puntas sin seguir el orden temporal; al final, J6 solo cabe en B1 después de su almuerzo. Este resultado y la asignación de cada vuelta coinciden con la salida de python `-m sched demo --traffic T2` y con la hoja *Instancia manual* del Excel.

8. Plan de pruebas y validación

8.1 Validación de restricciones

El módulo `validator.py` verifica cada solución sin usar las estructuras internas de los algoritmos. Recalcula: (1) que cada vuelta aparezca una sola vez, asignada o no atendida; (2) que el bus sea de la ruta; (3) la ventana; (4) la jornada; (5) que la duración coincida con $p_i(s_i)$; (6) que el almuerzo sea único, de 120 min y dentro de su ventana; y (7) que no haya solapamientos. Cada ejecución desde la línea de comandos lo aplica y termina con error si encuentra violaciones.

8.2 Casos de prueba

Tipo	Caso	Resultado esperado
Básico	Instancia manual con LS y LPT en T0, T1 y T2	Soluciones válidas que atienden las 8 vueltas
Básico	Cargas máximas de la instancia manual	T0: 270 (LS y LPT); T2: 329.0 (LS) y 333.2 (LPT)
Básico	Asignación de cada vuelta en el ejemplo manual	Coincide con las trazas de la sección 7
Límite	Ventana de ancho cero ($r_i = d_i$)	Salida exactamente en r_i
Límite	Vuelta que termina exactamente a las 22:00	Se acepta
Límite	Un solo bus ($m = 1$)	Solución válida
Límite	Duraciones iguales sin tráfico	LPT coincide con LS
Adverso	3 vueltas simultáneas y 1 bus	2 vueltas no atendidas, sin violaciones
Adverso	Peor caso clásico de LPT $\{3, 3, 2, 2\}$ h con $m = 2$	LPT obtiene 7 h (el óptimo es 6 h)
Ventanas	80 vueltas con ventanas y tráfico T2	Toda salida dentro de su ventana
Tráfico	Duración asignada en la instancia manual	Igual a $p_i(s_i)$ del perfil
Tráfico	Propiedad FIFO en pares aleatorios	$s_1 \leq s_2 \Rightarrow \text{llegada}_1 \leq \text{llegada}_2$
Tráfico	Cálculo de $p_{j1}(06:00)$ en T2 y media de T1	103.67 min; media de T1 = 1
Tráfico	Carga total con T2 frente a T0	Mayor con T2
Almuerzo	Vuelta que choca con un almuerzo fijo	Queda no atendida
Almuerzo	Vuelta que choca con un almuerzo flexible	El almuerzo se desplaza y la vuelta se atiende
Almuerzo	Almuerzo de cada bus	Dentro de su ventana y de 120 min
Validación de restricciones	Solución alterada a propósito	El validador la rechaza
Validación de restricciones	Cota inferior frente a $L_{\max}$	$LB \leq L_{\max}$
Validación de restricciones	400 vueltas y 60 buses	Solución válida; todas las vueltas contabilizadas
Validación de datos	Lectura del Excel del proyecto	35 rutas, 4 733 vueltas, coinciden con la regla general

Tabla 17. Plan de pruebas de la Entrega 1 (automatizado con pytest).

8.3 Verificación de las salidas

1. **Factibilidad:** el validador recalcula todas las restricciones en cada ejecución.

2. **Cálculo a mano:** la instancia de la sección 7 y la calculadora del Excel coinciden con el programa.
3. **Calidad:** la carga máxima se compara con la cota inferior LB de las vueltas atendidas.
4. **Instancias nuevas:** cualquier instancia escrita en el formato de la sección 6 se resuelve y valida con `python -m sched run --instance archivo.json`.

La comparación con la solución exacta en instancias pequeñas corresponde a los mecanismos de referencia de la sección 5.6, previstos para una etapa posterior.

9. Metodología experimental

9.1 Preguntas e hipótesis

#	Pregunta	Hipótesis
Q1	¿Qué estrategia atiende más vueltas y obtiene menor $L_{\max}$?	H1: sin tráfico LS y LPT coinciden (las vueltas de una ruta duran lo mismo); con tráfico LS supera a LPT.
Q2	¿Cómo afecta el tráfico a la cantidad de vueltas atendidas?	H2: T1 tiene un efecto moderado; T2 reduce marcadamente la atención.
Q3	¿Qué rutas son más difíciles de programar?	H3: las vueltas no atendidas se concentran en las rutas de mayor saturación p .

Tabla 18. Preguntas de evaluación e hipótesis de la Entrega 1.

9.2 Experimento E1: instancias, variables y métricas

- **Instancias:** las 35 rutas experimentales (4 733 vueltas y 1 080 buses).
- **Variables:** algoritmo {LS, LPT} $\times$ perfil de tráfico {T0, T1, T2}, es decir, 6 configuraciones por ruta.
- **Métricas principales** (función objetivo): vueltas no atendidas U y carga máxima $L_{\max}$.
- **Métricas complementarias:** brecha de $L_{\max}$ respecto de la cota inferior de las vueltas atendidas, desbalance $L_{\max} - L_{\min}$, sobrecosto por tráfico $(\sum p_i(s_i) - \sum \bar{p}_i) / \sum \bar{p}_i$, tiempo de ejecución y violaciones detectadas por el validador.

La metodología global del proyecto prevé, en etapas posteriores, la comparación con los mecanismos de referencia en instancias pequeñas, el análisis de escalabilidad y la evaluación de la robustez ante la variabilidad del tráfico. Esos análisis no forman parte de esta entrega.

9.3 Entorno y procedimiento reproducible

- Hardware: Intel Xeon 2.10 GHz (2 núcleos), Linux. Software: Python 3.13, pandas 3.0, matplotlib 3.11, openpyxl 3.1, pytest.
- Procedimiento: `python -m sched build` genera las 35 instancias; `python experiments/run_experiments.py` ejecuta E1 y escribe `results/E1_35_rutas.csv`, `results/E1_resumen.csv` y las figuras; `python experiments/build_excel.py` genera el Excel de la entrega.
- Los algoritmos son deterministas: repetir el procedimiento produce los mismos resultados (salvo los tiempos de ejecución).

10. Prototipo y resultados preliminares

10.1 Prototipo

Repositorio: [completar con la URL de GitHub del grupo]

```

transporte-cusco-scheduling/
├── src/sched/          traffic · model · builder · validator · metrics · viz · cli
│   └── algorithms/    greedy.py (LS, LPT) · bnb.py, cpsat.py (referencia, etapa
posterior)
├── tests/              pytest: test_ls_lpt · test_traffic · test_datos ·
test_referencia
├── experiments/        run_experiments.py (E1) · build_excel.py · demo_prototipo.ipynb
├── data/               raw/ (Excel, 35 rutas) · instances/ (JSON) · traffic/ (perfiles
JSON)
├── results/            E1_35_rutas.csv · E1_resumen.csv · figuras · Excel de la entrega
└── docs/               plan de aprendizaje, bitácoras, contribuciones

```

El prototipo **carga una instancia, genera las vueltas, ejecuta LS o LPT, valida la solución y produce una salida verificable**. Salida real de la ejecución:

```

$ python -m sched build
35 instancias + DEMO escritas en data/instances; perfiles en data/traffic

$ python -m sched run --route RTI-01 --alg ls --traffic T1 --gantt
RTI-01 (E.T. SAYLLA S.A.) · 35 buses · 103 vueltas · algoritmo LS · tráfico T1
  L_max = 7.772 h | L_min = 5.207 h | desbalance = 2.565 h | cota inf. = 7.063 h | gap
= 10.0 %
  asignados = 103/103 | utilización = 52.5 % | sobrecosto tráfico = 0.7 % | tiempo =
12.8 ms
  VALIDACIÓN: OK (0 violaciones)
  asignaciones -> results/RTI-01_LS_T1.csv

$ python -m sched demo --traffic T2          # instancia manual con LS y LPT
$ python -m pytest -q
36 passed

```

Una instancia conocida (la de la sección 7) o una nueva escrita en el formato JSON de la sección 6 se resuelve con `python -m sched run --instance archivo.json --alg ls (o --alg lpt)`, y el validador confirma su factibilidad.

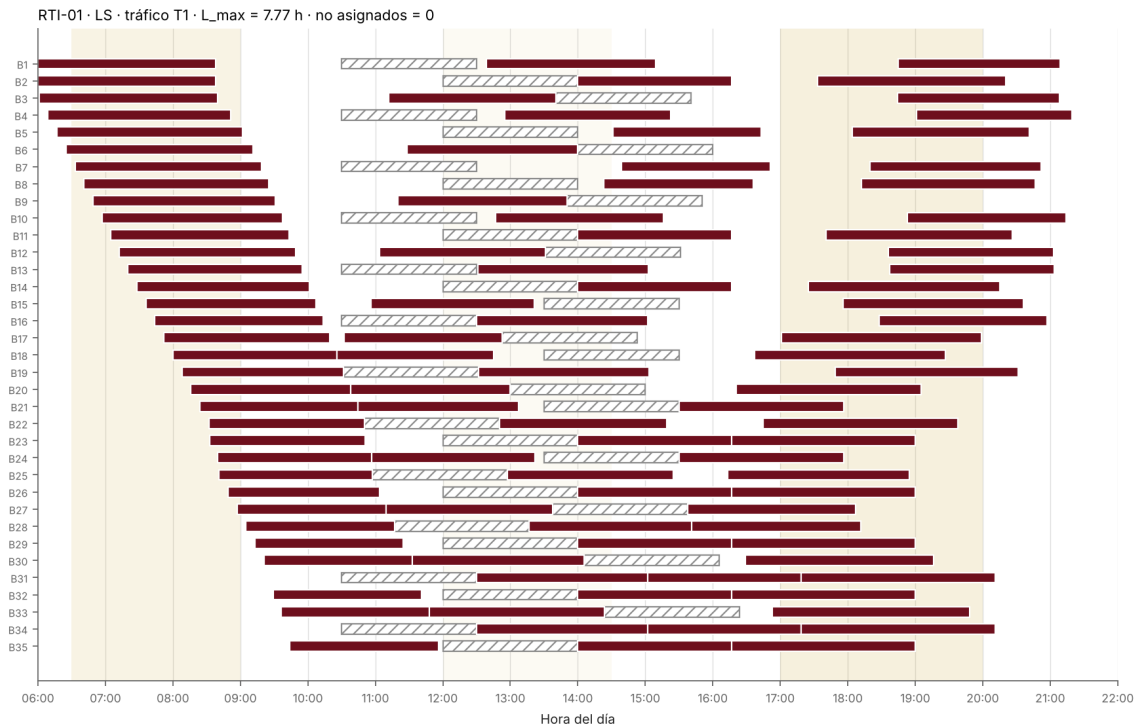


Figura 5. Programa de RTI-01 (35 buses, 103 vueltas) con List Scheduling y tráfico T1.

10.2 Resultados preliminares de E1

Tráfico	Algoritmo	Vueltas no atendidas	Rutas completas	$L_{\max}$ medio (h)	Brecha vs LB	Desbalance (h)	Sobrecosto por tráfico
T0	LS = LPT	87 (1.8 %)	18	9.15	10.3 %	1.91	0 %
T1	LS	155 (3.3 %)	8	8.96	13.6 %	1.65	+0.8 %
T1	LPT	293 (6.2 %)	10	9.13	19.4 %	2.53	+0.8 %
T2	LS	255 (5.4 %)	4	9.87	14.2 %	1.92	+13.4 %
T2	LPT	547 (11.6 %)	4	10.04	24.1 %	3.41	+13.7 %

Tabla 19. Resultados agregados de E1 sobre las 35 rutas (4 733 vueltas; brecha y desbalance promediados por ruta; 0 violaciones en todas las soluciones).

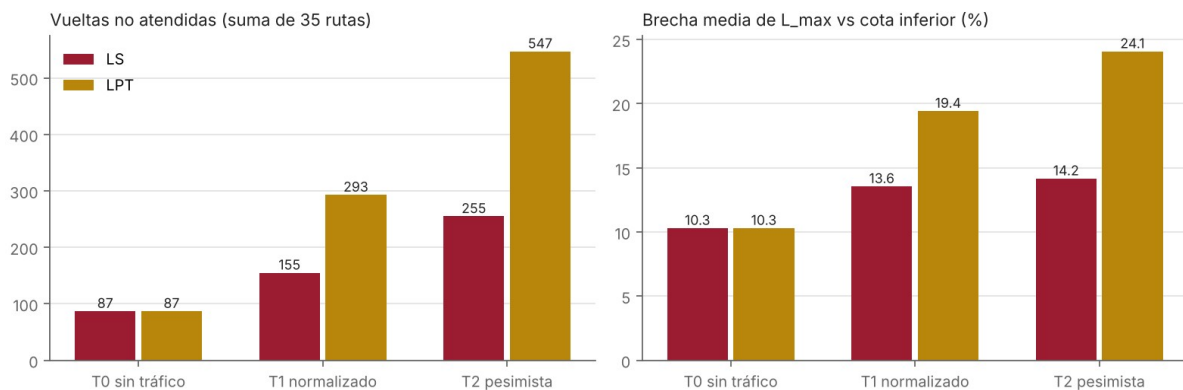


Figura 6. Vueltas no atendidas y brecha media respecto de la cota inferior, por escenario de tráfico.

- **Sin tráfico, LS y LPT coinciden.** Las vueltas de una ruta tienen la misma duración base, así que el orden de LPT empata y se desempata por r_i , que es el orden de LS.
- **Con tráfico, LS obtiene mejores resultados en las instancias evaluadas.** En T1 deja 155 vueltas sin atender frente a 293 de LPT y es igual o mejor en 26 de 35 rutas; en T2 es mejor en 31 de 35. Su desbalance es 35–44 % menor.
- **El tráfico reduce la atención:** con LS, la proporción de vueltas atendidas pasa de 98.2 % (T0) a 96.7 % (T1) y 94.6 % (T2). En T1 la conducción total casi no cambia (+0.8 %), porque el perfil solo redistribuye el tiempo; en T2 aumenta 13.4 %.
- **Las vueltas no atendidas sin tráfico se concentran en rutas saturadas:** RTU-22 (21; $p = 1.02$), RTI-02 (15; $p = 0.87$), RTU-01 (12; $p = 0.79$) y RTU-05 (7; $p = 0.82$).
- **Tiempo de ejecución:** cada configuración resuelve las 35 rutas en menos de 0.7 s (unos 20 ms por ruta).

10.3 Contraste con las hipótesis

Hipótesis	Resultado preliminar	Estado
H1: LS = LPT sin tráfico; LS mejor con tráfico	Idénticos en T0; LS mejor en T1 y T2	Se cumple en las instancias evaluadas
H2: efecto moderado de T1 y marcado de T2	+68 vueltas no atendidas en T1 (+1.4 pp) y +168 en T2 (+3.5 pp) respecto de T0	Se cumple
H3: no atendidas concentradas en rutas de alta p	Las 4 rutas con más vueltas no atendidas tienen $p \geq 0.79$	Se cumple

Tabla 20. Contraste preliminar de hipótesis.

11. Discusión y limitaciones

11.1 Interpretación

La garantía teórica de LPT supone que cualquier trabajo puede ejecutarse en cualquier momento, de modo que solo importa su tamaño. Con ventanas de inicio, en cambio, el orden de procesamiento determina la factibilidad. Cuando las ventanas son estrechas, el problema se parece a la partición de intervalos, donde procesar los intervalos por hora de inicio es la estrategia óptima en número de máquinas (Kolen et al., 2007). LS conserva ese orden temporal; LPT lo rompe al insertar primero vueltas de la tarde y dejar huecos fragmentados en la mañana.

Con tráfico, además, la vuelta "más larga" es la que cae en hora punta, así que LPT termina ordenando por congestión y no por tamaño, y concentra vueltas de punta en los mismos buses. El mayor desbalance de LPT lo confirma (2.53 h frente a 1.65 h en T1).

El ejemplo manual muestra también que la hora de salida dentro de la ventana influye en el tiempo de viaje (J4 tarda 103.85 min si sale a las 08:00 y 97.62 min si sale a las 08:27). LS y LPT eligen siempre la salida factible más temprana; cuantificar cuánto se gana eligiendo otra salida corresponde a la comparación con los mecanismos de referencia prevista para una etapa posterior.

11.2 Limitaciones y dificultades pendientes

- **Parámetros experimentales.** Las ventanas, los turnos de almuerzo y los factores de tráfico son supuestos definidos por el grupo; los resultados comparan algoritmos y no predicen la operación real.
- **Datos de referencia.** La tabla de 2020 es un documento secundario y representa una jornada típica.
- **Calidad de las heurísticas.** La brecha de LS respecto de la cota inferior es de 10–14 %. La cota puede ser poco ajustada, por lo que la distancia real al óptimo debe medirse con los mecanismos de referencia.
- **Variabilidad del tráfico.** En esta entrega el tráfico es determinístico por escenario; el efecto de las variaciones aleatorias de los tiempos de viaje no se ha evaluado.
- **Rutas saturadas.** En rutas con ρ cercano o mayor que 1, ninguna asignación puede atender todas las vueltas; el resultado depende de la capacidad de la flota y no solo del algoritmo.

12. Conclusiones

1. Se formuló la asignación de vueltas a buses como un problema de máquinas paralelas idénticas con ventanas de inicio, almuerzo y tiempos de viaje $p_j(s_j)$ dependientes de la hora de salida, con objetivo lexicográfico: primero maximizar las vueltas atendidas y luego minimizar la carga máxima $L_{\max}$.
2. El tráfico se incorporó mediante un perfil de congestión por franjas que cumple la propiedad FIFO, evaluado en tres escenarios experimentales.
3. Los resultados preliminares muestran que, sin tráfico, List Scheduling y LPT producen la misma solución, y que con tráfico LS atiende más vueltas y equilibra mejor la carga en las instancias evaluadas (en T1, 155 vueltas no atendidas frente a 293).
4. En las 35 rutas evaluadas, el tráfico pesimista reduce la proporción de vueltas atendidas con LS de 98.2 % a 94.6 % y aumenta la conducción total en 13.4 %.
5. La saturación de la flota (ρ) explica buena parte de las vueltas no atendidas; en RTU-22 la demanda de horas supera la capacidad de la flota.
6. El prototipo permite verificar el modelo de extremo a extremo: genera las instancias con una regla general, ejecuta LS y LPT, valida todas las restricciones y produce resultados reproducibles, lo que muestra la viabilidad del proyecto.

13. Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). MIT Press.
- Garey, M. R. y Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W. H. Freeman.
- Gawiejnowicz, S. (2008). *Time-dependent scheduling*. Springer.
- Google. (2024). *OR-Tools: CP-SAT solver* [Documentación oficial].
<https://developers.google.com/optimization>
- Graham, R. L. (1966). Bounds for certain multiprocessing anomalies. *Bell System Technical Journal*, 45(9), 1563–1581.
- Graham, R. L. (1969). Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2), 416–429.
- Ibarra-Rojas, O. J., Delgado, F., Giesen, R. y Muñoz, J. C. (2015). Planning, operation, and control of bus transport systems: A literature review. *Transportation Research Part B: Methodological*, 77, 38–75.
- Ichoua, S., Gendreau, M. y Potvin, J.-Y. (2003). Vehicle dispatching with time-dependent travel times. *European Journal of Operational Research*, 144(2), 379–396.
- Kolen, A. W. J., Lenstra, J. K., Papadimitriou, C. H. y Spieksma, F. C. R. (2007). Interval scheduling: A survey. *Naval Research Logistics*, 54(5), 530–543.
- Land, A. H. y Doig, A. G. (1960). An automatic method of solving discrete programming problems. *Econometrica*, 28(3), 497–520.
- Lee, C.-Y. (1996). Machine scheduling with an availability constraint. *Journal of Global Optimization*, 9, 395–416.
- Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P. y Wießner, E. (2018). Microscopic traffic simulation using SUMO. En *21st IEEE International Conference on Intelligent Transportation Systems (ITSC)* (pp. 2575–2582).
- Pinedo, M. L. (2016). *Scheduling: Theory, algorithms, and systems* (5.^a ed.). Springer.
- Tabla de datos – Unidades de transporte – Flota operativa 2020. (s. f.). Scribd.
<https://es.scribd.com/document/553897271/TABLA-DE-DATOS-UNIDADES-DE-TRANSPORTE-FLOTA-OPERATIVA-2020>

Anexo A. Manual de instalación y uso

Requisitos: Python ≥ 3.10 , en Windows, Linux o macOS.

```
git clone <URL-del-repositorio> && cd transporte-cusco-scheduling
python -m venv .venv
source .venv/bin/activate                # Windows: .venv\Scripts\activate
pip install -r requirements.txt
pip install -e .

python -m sched build                    # genera data/instances/*.json (35 rutas + DEMO)
python -m sched profiles                 # muestra los perfiles T0, T1 y T2
python -m sched demo --traffic T2        # instancia manual con LS y LPT
python -m sched run --route RTU-05 --alg ls --traffic T1 --gantt
python -m sched run --route RTU-05 --alg lpt --traffic T1
python experiments/run_experiments.py    # experimento E1
python experiments/build_excel.py        # Excel de la entrega
python -m pytest -q                      # pruebas
```

Parámetro	Valores	Efecto
--route / --instance	código de ruta, DEMO o archivo JSON	Instancia a resolver
--alg	ls · lpt	Algoritmo de la Entrega 1
--traffic	T0 · T1 · T2 · archivo.json	Perfil de tráfico
--gantt, --out	—	Diagrama de Gantt y ruta del CSV de salida

Tabla 21. Opciones de la línea de comandos.

Anexo B. Contribuciones y bitácora

R = responsable, A = apoyo, V = revisión. La distribución se contrasta con el historial del repositorio.

Componente	Aguilar	Choque	Huacani	Moreano
Formulación y modelo de tráfico	R	V	V	A
Modelo de datos y línea de tiempo	R	A	V	
List Scheduling y LPT	A	R	V	
Validador, métricas y pruebas	A	A	R	A
Datos, Excel e instancias	V			R
Experimento E1 y gráficos	A		V	R
Ejemplo manual	A	R	R	V
Informe	R	A	A	A

Tabla 22. Matriz de contribuciones.

Cada integrante mantiene una bitácora individual en docs/bitacora/, con una fila por sesión de trabajo: fecha, conocimiento trabajado, actividad, fuente, lo aprendido, dónde se aplicó (archivo o commit), dificultad y solución, y horas.

Anexo C. Resultados por ruta

ρ = saturación de la flota; NA = vueltas no atendidas; $L_{\max}$ en horas. Datos completos en `results/E1_35_rutas.csv` y en la hoja *Resultados por ruta* del Excel.

Ruta	Buses	Vueltas	ρ	NA T0	LS NA T1	LS $L_{\max}$ T1	LPT NA T1	LPT $L_{\max}$ T1	LS NA T2	LPT NA T2
RTI-01	35	103	0.52	0	0	7.77	3	9.73	4	13
RTI-02	27	127	0.87	15	19	10.74	36	10.70	29	43
RTI-04	42	144	0.55	1	2	8.64	4	9.04	5	16
RTI-05	44	150	0.48	0	1	7.55	0	7.87	2	3
RTI-06	32	144	0.56	0	3	8.53	0	8.67	4	8
RTI-07	39	144	0.64	4	8	9.46	15	9.90	11	28
RTI-08	20	108	0.61	0	1	9.23	1	9.31	3	11
RTI-09	43	135	0.54	0	2	9.21	8	9.54	6	17
RTI-10	37	166	0.62	1	4	9.48	11	9.73	7	19
RTU-01	32	144	0.79	12	15	11.93	28	10.12	22	39
RTU-02	29	117	0.63	2	4	8.91	12	8.95	8	18
RTU-04	28	144	0.75	5	7	10.53	21	10.35	13	29
RTU-05	24	144	0.82	7	10	11.50	28	9.91	18	36
RTU-07	28	127	0.46	0	0	7.03	0	7.11	1	0
RTU-08	26	114	0.55	0	2	8.64	2	8.77	3	6
RTU-09	21	120	0.65	0	1	9.70	5	9.60	4	12
RTU-10	16	103	0.61	0	0	9.11	2	9.26	1	6
RTU-11A	21	103	0.42	0	0	6.22	0	6.13	0	0
RTU-11B	22	103	0.40	0	0	6.06	0	6.16	0	0
RTU-12	39	150	0.52	2	3	7.62	0	7.77	4	5
RTU-13	37	144	0.65	3	8	9.32	16	11.65	11	26
RTU-14	37	108	0.42	0	0	6.35	0	6.21	0	0
RTU-15	30	127	0.58	1	4	9.21	3	9.58	5	11
RTU-16	40	150	0.53	2	2	7.86	0	7.97	5	10
RTU-17	28	127	0.57	0	2	8.72	1	8.95	3	7
RTU-18	35	166	0.58	0	4	8.54	0	10.12	5	8
RTU-19	34	144	0.70	6	10	9.51	21	9.44	14	31
RTU-20	31	154	0.55	0	2	7.87	2	7.86	4	6
RTU-21	26	166	0.59	0	0	8.86	0	9.00	0	6
RTU-22	19	127	1.02	21	25	12.94	39	12.83	37	52
RTU-23	37	166	0.60	1	5	9.23	4	9.39	5	19
RTU-24	35	117	0.58	2	4	9.42	7	9.70	7	22
RTU-25	27	127	0.56	0	2	8.29	1	8.42	3	5

Ruta	Buses	Vueltas	ρ	NA T0	LS NA T1	LS $L_{\max}$ T1	LPT NA T1	LPT $L_{\max}$ T1	LS NA T2	LPT NA T2
RTU-26	33	154	0.66	2	5	9.79	16	10.00	9	23
RTU-27	26	166	0.65	0	0	9.73	7	9.84	2	12
Total	1080	4733		87	155		293		255	547

Tabla 23. Resultados por ruta (resaltadas: rutas con $\rho \geq 0.85$).